

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САУ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Техническое зрение»
Тема: Преобразование Хафа для поиска
прямых и окружностей

Студент гр. 2491

Недовизий А.И.

Преподаватель

Федоркова А.О.

Санкт-Петербург

2025

Цель работы: изучить методы поиска прямых и окружностей с помощью преобразования Хафа.

Задания к работе:

Задание 1:

Исправить изображение так, чтобы линии таблицы исчезли, а числа остались.

Задание 2:

Для выполнения этого задания используйте обработанное операцией размыкания изображение из работы 2. На нём найдите самый длинный отрезок и самую большую окружность.

Задание 3:

Для выполнения этого задания используйте картинки 4_5_1 - 4_5_6. Найдите любые три круглых дорожных знака.

Ход выполнения работы:

Задание 1:

Код программы приведен в листинге 1.

Листинг 1

```
import cv2
import numpy as np
from paths import *

image = cv2.imread(path4_1)
cv2.imshow("window", image)
cv2.waitKey()

image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# Применение Гауссова размытия для уменьшения шума и улучшения обнаружения границ
blurred = cv2.GaussianBlur(image, (5, 5), 0)
# Обнаружение границ с помощью детектора границ Canny
edges = cv2.Canny(blurred, 50, 150, apertureSize=3)

lines = cv2.HoughLines(
    edges, 1, np.pi / 180, 200
) # по сути рисуем черными линиями поверх белых, так что они пропадают с изображения
for line in lines:
    rho, theta = line[0]
    a = np.cos(theta)
    b = np.sin(theta)
    x0 = a * rho ##находим точку пересечения перпендикуляра и самой линии
    y0 = b * rho ## если вектор перпендикуляра (a,b) то вектор прямой - (-b, a)
    x1 = int(
        x0 + 1000 * (-b)
    ) ## откладываем от точки пересечения 1000 пикселей по прв одно направление
    y1 = int(y0 + 1000 * (a))
    x2 = int(x0 - 1000 * (-b)) ## откладываем 1000 пикселей в другое направление
    y2 = int(y0 - 1000 * (a))
    cv2.line(
        edges, (x1, y1), (x2, y2), (0, 0, 0), 2
    ) ## рисуем прямую по этим двум точкам

# Отображение результата
cv2.imshow("only numbers", edges)
```

```
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Задание 2:

Код приведен в листинге 2

Листинг 2

```
import cv2
import numpy as np
from paths import *

path1 = r"/Users/new/Desktop/course4labs/cv/lab2/2-1.jpg"

def dist(point: list):
    return ((point[0] - point[2]) ** 2 + (point[1] - point[3]) ** 2) ** (1 / 2)

def create_black(image):
    img = np.zeros_like(image)
    return img

def init(path):
    image = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    show_and_wait(winname="sourcePic", image=image)
    # clear from isnides given image
    clear_img = cv2.morphologyEx(
        image, op=cv2.MORPH_CLOSE, kernel=np.ones((3, 3), dtype=np.uint8), iterations=1
    )
    image_reverse = 255 - clear_img
    show_and_wait(winname="reversePic", image=image_reverse)
    blurred = cv2.medianBlur(image_reverse, ksize=3)
    edges = cv2.Canny(blurred, 100, 200, apertureSize=3, L2gradient=True)
    return image, edges
```

```

def draw_longest_line(edges, image):
    lines = cv2.HoughLinesP(
        edges, rho=1, theta=np.pi / 360, threshold=80, minLineLength=30, maxLineGap=15
    )
    lines = np.reshape(lines, shape=(11, 4))
    black = create_black(image)
    mapped_lines = list(map(lambda line: dist(line), lines))
    x1, y1, x2, y2 = lines[np.argmax(mapped_lines)]
    cv2.line(black, (x1, y1), (x2, y2), (255, 255, 255), 2)
    return black

def show_and_wait(image, winname="windowww"):
    cv2.imshow(winname, image)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

def find_biggest_circle(path, canvas):
    pic = cv2.imread(path, cv2.IMREAD_COLOR)
    pic_g = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    pic_f = cv2.GaussianBlur(255 - pic_g, (3, 3), sigmaX=1)
    a = pic.shape[0]
    circles = cv2.HoughCircles(
        image=pic_f,
        method=cv2.HOUGH_GRADIENT,
        dp=1, ## разрешение аккумулятора
        minDist=a // 20, ## минимальное расстояние между окружностями(центрами)
        param1=120, ## верхний трешхолд детектора Кенни (нижний берется как 1/2 от верхнего)
        param2=80, ## порог аккумулятора
        minRadius=a // 100, ## минимальный и максимальный радиус
        maxRadius=0,
    )
    circles = np.reshape(circles, shape=(-1, 3))
    biggest_circle = circles[np.argmax(circles[:, 2])]
    center = int(biggest_circle[0]), int(biggest_circle[1])
    radius = int(biggest_circle[2])
    cv2.circle(canvas, center, radius, (255, 255, 255), 3)
    show_and_wait(winname="result", image=canvas)
    show_and_wait(winname="original", image=pic_g)

```

```
def main():
    image, edges = init(path1)
    black = draw_longest_line(edges, image)
    show_and_wait(black, "longest line")
    find_biggest_circle(path1, canvas=black)

if __name__ == "__main__":
    main()
```

Задание 3:

Код приведен в листинге 2

Листинг 3

```
import cv2
import numpy as np
from paths import *

def show_and_wait(image, winname="windowww"):
    cv2.namedWindow(winname, cv2.WINDOW_NORMAL)
    cv2.imshow(winname, image)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

def find_circle(path, canvas):
    pic = cv2.imread(path, cv2.IMREAD_COLOR)
    pic_g = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    pic_f = cv2.GaussianBlur(255 - pic_g, (3, 3), sigmaX=2)
    a = pic.shape[0]
    circles = cv2.HoughCircles(
        image=pic_f,
        method=cv2.HOUGH_GRADIENT,
        dp=1, ## разрешение аккумулятора
        minDist=a // 20, ## минимальное расстояние между окружностями(центрами)
        param1=240, ## верхний трешхолд детектора Кенни (нижний берется как 1/2 от верхнего)
        param2=70, ## порог аккумулятора
        minRadius=a // 100, ## минимальный и максимальный радиус
        maxRadius=0,
```

```

)
circles = np.reshape(circles, shape=(-1, 3))
biggest_circle = circles[np.argmax(circles[:, 2])]
center = int(biggest_circle[0]), int(biggest_circle[1])
radius = int(biggest_circle[2])
cv2.circle(canvas, center, radius, (255, 255, 255), 3)
show_and_wait(winname="result with a circled sign", image=canvas)
show_and_wait(winname="original", image=pic_g)

def create_black(image):
    img = np.zeros_like(image)
    return img

def init(path):
    image = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    show_and_wait(winname="sourcePic", image=image)
    blurred = cv2.GaussianBlur(src=image, ksize=(3, 3), sigmaX=3, sigmaY=3)
    edges = cv2.Canny(blurred, 100, 200, apertureSize=3, L2gradient=True)
    return image, edges

def main():
    for path in [path2, path5, path4]:
        image, edges = init(path)
        show_and_wait(winname="edges", image=edges)
        black = create_black(image)
        find_circle(path=path, canvas=black)

if __name__ == "__main__":
    main()

```

Вывод: в результате лабораторной работы были освоены основные способы обнаружения линий и окружностей. Был изучен алгоритм Хафа, для обнаружения линий.